

ASSIGNMENT – 1

Basic Queries

1. Display all documents from the **products** collection.

```
db["products"].find()
```

2. Display all products that belong to the **Electronics** category.

```
db.products.find({"category": "Electronics"});
```

3. Display all products that are available in **Hyderabad**.

```
db.products.find({"city": "Hyderabad"});
```

4. Display products whose **price** is greater than 30000.

```
db.products.find({price: {$gt: 30000}});
```

5. Display products whose **price** is less than 20000.

```
db.products.find({price: {$lt: 20000}});
```

Filtering Queries

6. Display products whose **price** is between 10000 and 50000.

```
db.products.find({  
  price:  
    {$lt: 50000,  
     $gt: 10000}  
});
```

7. Display all products that belong to the **Furniture** category.

```
db.products.find({  
  "category": "Furniture"  
});
```

8. Display products that belong to the **Electronics** category and are located in **Hyderabad**.

```
db.products.find({  
  "category": "Electronics",  
  "city": "Hyderabad"  
});
```

9. Display products that belong to **Hyderabad** or **Bangalore**.

```
db.products.find({  
  city: { $in: ["Hyderabad", "Bangalore"] }  
});
```

10. Display products that **do not** belong to the **Furniture** category.

```
db.products.find({  
  category: { $ne: "Furniture" }  
});
```

Projection Queries

11. Display **only the name and price** of all products.

```
db.products.find({}, {name:1, price:1, _id:0});
```

12. Display **name, category, and city** for all products.

```
db.products.find({}, {name:1, category:1, city:1, _id:0});
```

Sorting Queries

13. Display all products **sorted by price in ascending order**.

```
db.products.find().sort({price: 1});
```

14. Display all products **sorted by price in descending order**.

```
db.products.find().sort({price: -1});
```

Limit Queries

15. Display the **top 3 most expensive products**.

```
db.products.find().sort({price: -1}).limit(3);
```

16. Display the **2 cheapest products**.

```
db.products.find().sort({price: 1}).limit(2);
```

Skip Queries

17. Display all products ****skipping the first two documents****.

```
db.products.find().skip(2);
```

Additional Filtering

18. Display products whose ****stock is greater than 10****.

```
db.products.find({stock: {$gt: 10}});
```

19. Display products whose ****stock is less than or equal to 10****.

```
db.products.find({stock: {$lte:10}});
```

20. Display ****Electronics products with price greater than 40000****.

```
db.products.find({category: "Electronics",price: {$gt: 40000}});
```

Update Operations

21. Update the ****price of the product named "Laptop"****.

```
db.products.updateMany(  
  {name: "Laptop"},  
  {$set: {price:40000}}  
)
```

22. Add a new field called ****discount = 10**** for all ****Electronics products****.

```
db.products.updateMany(  
  {category: "Electronics"},  
  {$set: {discount: 10}}  
)
```

Delete Operations

23. Delete the product whose **name** is "Printer".

```
db.products.deleteMany(  
  {name: "Printer"}  
);
```

24. Delete all products belonging to the **Furniture** category.

```
db.products.deleteMany(  
  {category: "Furniture"}  
);
```

Counting Documents

25. Count the **total number of products**.

```
db.products.countDocuments({})
```

26. Count the number of products belonging to the **Electronics** category.

```
db.products.countDocuments({category: "Electronics"});
```

Aggregation Queries

27. Calculate the **total stock available for each category**.

```
db.products.aggregate([
  {
    $group:{
      _id : "$category",
      totalStock: {$sum: "$stock"}
    }
  }
])
```

28. Calculate the **average price of products for each category**.

```
db.products.aggregate([
  {
    $group:{
      _id : "$category",
      AveragePrice: {$avg: "$price"}
    }
  }
])
```

29. Find the **maximum product price**.

```
db.products.aggregate([
  {
    $group:{
      _id : null,
      MaximumPrice: {$max: "$price"}
    }
  }
])
```

30. Calculate the ****total inventory value**** (price × stock of all products).

```
db.products.aggregate([
  {
    $project:{
      itemTotal: {$multiply: ["$price", "$stock"]}
    }
  },
  {
    $group:{
      _id : null,
      totalInventoryValue: {$sum: "$itemTotal"}
    }
  }
])
```
